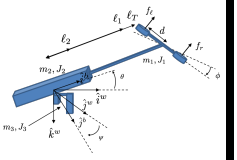
Hummingbird Manual

Introduction to Feedback Control

USING DESIGN STUDIES



RANDAL W. BEARD
MARC D. KILLPACK
WITH
MARK PETERSEN
DEVON MORRIS

Updated August 17, 2026

TOC

Contents

1 Introduction 1
Lab: hummingbird 4

I Simulation Models 5
2 Kinetic Energy of Mechanical Systems 7
3 The Euler Lagrange Equations 17

II Hummingbird Design Models 23
4 Equilibria and Linearization 25
5 Transfer Function Models 33
6 State Space Models 35

III PID Control Design 37
7 Pole Placement for Second Order Systems 39
8 Design Strategies for Second-Order Systems 43
9 System Type and Integrators 49
10 Digital Implementation of PID Controllers 51

IV Observer-based Control Design 55
11 Full-state Feedback with Integral Control 57
12 Observers 61
13 Disturbance Observers 65
a Parameters of the Hummingbird 67

Appendices 66

1 Introduction

The objective of this document is to provide a detailed introduction to the BYU hummingbird including laboratory assignments that are intended to complement an introductory course in feedback control. The hummingbird is a three degree-of-freedom helicopter with complicated nonlinear dynamics, but the dynamics are amenable to linear analysis and design. The hummingbird has been designed with encoders on each joint that allow full-state feedback (requiring taking numerical derivatives or using an observer to obtain angular rates).

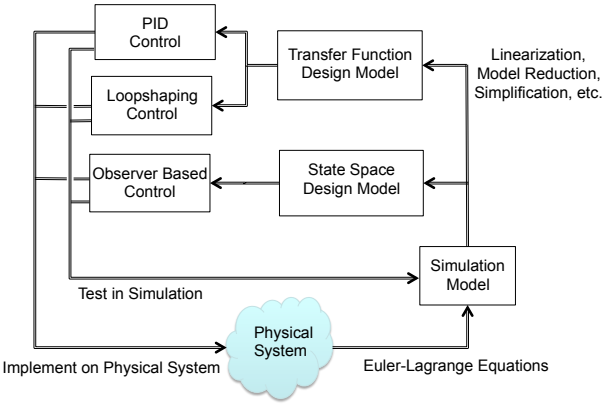


Figure 1-1: The design process. Using physics models the physical system is modeled with nonlinear differential equations resulting in the simulation model. The simulation model is simplified to create design models which are used for the control design. The control design is then tested and debugged in simulation and finally implemented on the physical system.

The design process for control systems is shown in Figure 1-1. For the hummingbird, the system to be controlled is an electromechanical system with actu-

ators (motors) and sensors (e.g. encoders). The first step in the design process is to model the physical system using nonlinear differential equations. While approximations and simplifications will be necessary at this step, the hope is to capture in mathematics, all of the important characteristics of the physical system. The mathematical model of the system is usually obtained using Euler-Lagrange method, Newton's method, or other methods from physics. The resulting model is usually too complex for design. However, this model will be used to test later designs in simulation and is therefore called the Simulation Model, as shown in Figure 1-1. To facilitate design, the simulation model is simplified and usually linearized to create lower-order (and usually linear) design models. For any physical system, there may be multiple design models that capture certain aspects of the design process. For the hummingbird, we will decompose the motion into longitudinal (pitching) motion and lateral (rolling and yawing) motion and we will have different design models for each type of motion. As shown in Figure 1-1, the design models are used to develop control designs. The control designs are then tested against the simulation, which sometimes requires that the design models be modified or enhanced if they have not captured the essential features of the system. After the control designs have been thoroughly tested in simulation, they are implemented on the physical system and are again tested and debugged, sometimes requiring that the simulation model be modified to more closely match the physical system.

The physical system or "hardware" requires either low-level C-code that is compiled and uploaded to a micro-controller (like an Arduino) to interface with actuators and sensors, or serial communication between Python and the micro-controller instead. In either case, the micro-controller communicates with the Electronic Speed Controllers (ESCs) to control the speed of the propellers, and these motor commands are the "inputs" into the system. The micro-controller also reads the angle data from encoders that measure roll, pitch, and yaw of the system as "outputs." These outputs are what we use at each time step to estimate error for our feedback loop.

It will be important for students to follow all the directions found in this manual (and instructions from TAs) as it will ensure that the hummingbird hardware will run in a safe manner and last longer for future students.

The purpose of these notes, and the associated laboratory assignments, is to walk you through the design cycle for the hummingbird. We will consider multiple control design methodologies (e.g. PID and observer-based control). We believe that visualizing the physical system is an important part of the simulation model. The simulation model has been partially implemented in Python using PyQt for visualization in 3D, and this code is found in the course code repository. Students will complete the simulation as part of the first few labs.

The 3D animation of the hummingbird that we provide (and any visualization of a control system) needs to behave in a dynamically accurate manner. The equations of motion for the hummingbird are derived using the Euler-Lagrange method in Chapters 2 and 3. Implementing the equations of motion in Python requires knowledge of numerical integration of ordinary differential equations (ODEs).

Therefore, a simulator template has been created for the students. This code will handle the integration of the equations of motion of the hummingbird and students will be required to find the equations of motion. Understanding the equations of motion and implementing them comprises Chapters 2 and 3. Support files for this approach are found on the control book website. Once the simulation model is complete, design models are developed in Chapters 4, 5, and 6. Since the design models are in terms of force and torque, the force and torque need to be converted to a left and right motor pulse width modulation command.

Developing physically realizable design specifications is an important part of the design process and described in Chapter 7. The remainder of the chapters introduce different methods to control the hummingbird according to the design specifications.

This book is used in conjunction with the Control Book website. The website contains many useful files such as the Design Study solutions for the Single Link Robot Arm, Inverted Pendulum, and Satellite Altitude Control.

H Hummingbird Lab Assignments

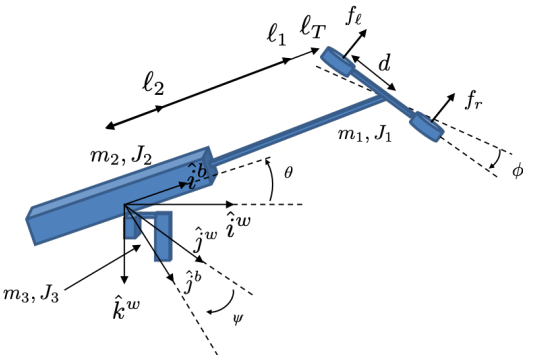


Figure 1-2: The hummingbird system

A descripton of the hummingbird can be found in Appendix a

Laboratory Assignments:

H.1 Intro to hummingbird.
H.2 Kinetic energy.
H.3 Equations of Motion.
H.4 Linearize equations of motion.
H.5 Transfer function model.
H.6 State space model.
H.7 Pole placement using PD.
H.8 Second order design.
H.9 Integrators and system type.
H.10 Digital PID.
H.11 Full state feedback.
H.12 Full state with integrator.
H.13 Observer based control.
H.14 Disturbance observer.
H.15 Frequency Response.
H.16 Loop gain.
H.17 Stability margins.
H.18 Loopshaping design.

Lab Assignment H.1

1. Read Chapter 1 and Appendix a to better understand the hummingbird and the process that we will follow to design controllers and test them in the subsequent labs.

Part I

Simulation Models

2

Kinetic Energy of Mechanical Systems

The objective of this chapter is to develop the equations that describe the kinetic energy of the hummingbird. The physical parameters of the hummingbird and a description of each can be found in Appendix a.

2.1 Kinetic Energy

A simplified diagram of the hummingbird is shown in Figure 2-1,

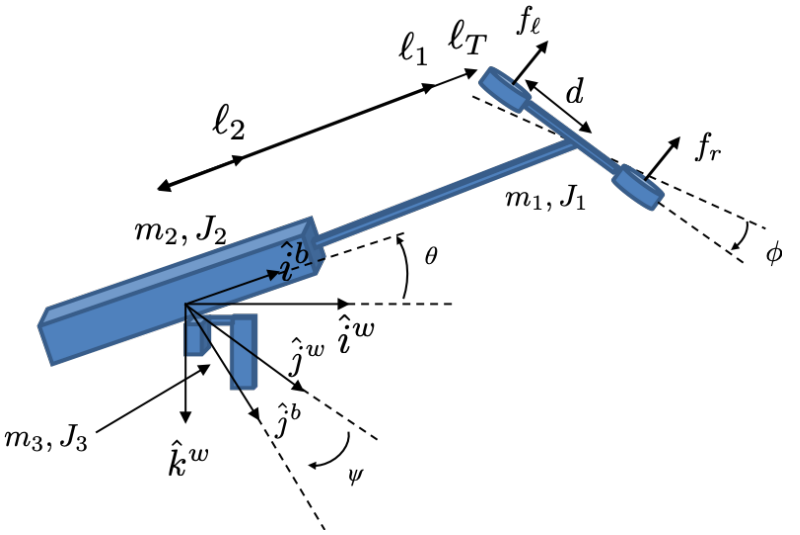


Figure 2-1: A simplified diagram of the hummingbird.

Since there are three different objects that move independently of each other,

we will model the hummingbird as consisting of three masses, as shown in Figure 2-1. The first mass, labeled m_1, J_1 in Figure 2-1 is the final rod and the head of the hummingbird, including the motors, propellers, and shrouds. The second mass, labeled m_2, J_2 in Figure 2-1 is the main body of the hummingbird, the third mass labeled m_3, J_3 in Figure 2-1 consists of the metal block that connects the main body and also the control board.

The world coordinate frame (or inertial frame) is fixed to the point where the third body spins about the z^w-axis. The $x^w - y^w$ plane is horizontal with respect to the ground, and the world z^w-axis points toward the center of the earth. The body-fixed coordinate frame (or frame that moves body 3) is defined so that center of the frame corresponds to the center of the world frame, and if an observer were sitting at the center and facing along the main shaft of the hummingbird, the x^b-axis would point forward, along the shaft of the hummingbird, the y^b axis would point to the right, and the z^b axis would point down (as shown in Figure 2-1).

2.1.1 Rotation matrices and transformations between coordinate frames

To fully understand the derivation of the equations of motion for the hummingbird requires an understanding of transformations between coordinate frames, and how to express a moving point (the center of mass) in an inertial frame.

One coordinate frame is transformed into another through two basic operations: rotation and translation. In this section we will describe rotation matrices and their use in transforming between coordinate frames, and we will also describe the specific coordinate frames used for the hummingbird.

2.2 Rotation Matrices

We begin by considering the two coordinate frames shown in Figure 2-2. The vector $\mathbf{p}$ can be expressed in both the $\mathcal{F}^0$ frame (specified by $(\hat{i}^0, \hat{j}^0, \hat{k}^0)$) and in the $\mathcal{F}^1$ frame (specified by $(\hat{i}^1, \hat{j}^1, \hat{k}^1)$). This is the same vector in either case, but expressed in a different frame. In the $\mathcal{F}^0$ frame we describe this as

$$\mathbf{p} = p_x^0 \hat{i}^0 + p_y^0 \hat{j}^0 + p_z^0 \hat{k}^0.$$

Alternatively in the $\mathcal{F}^1$ frame we have

$$\mathbf{p} = p_x^1 \hat{i}^1 + p_y^1 \hat{j}^1 + p_z^1 \hat{k}^1.$$

The vector sets $(\hat{i}^0, \hat{j}^0, \hat{k}^0)$ and $(\hat{i}^1, \hat{j}^1, \hat{k}^1)$ are each mutually perpendicular sets of unit basis vectors.

One way to write the matrix that describes the rotation between the two frames is to set these two expressions equal to each other, which gives

$$p_x^1 \hat{i}^1 + p_y^1 \hat{j}^1 + p_z^1 \hat{k}^1 = p_x^0 \hat{i}^0 + p_y^0 \hat{j}^0 + p_z^0 \hat{k}^0.$$

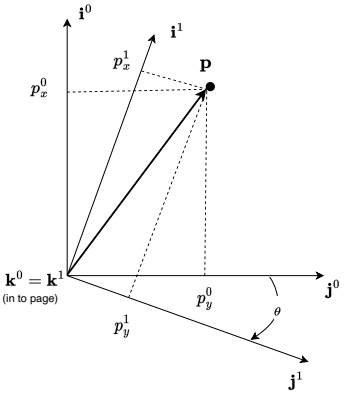


Figure 2-2: Rotation in 2D

Taking the dot product of both sides with $\hat{i}^0$, $\hat{j}^0$, and $\hat{k}^0$ respectively, and stacking the result into matrix form gives

$$\mathbf{p}^0 \triangleq \begin{pmatrix} p_x^0 \\ p_y^0 \\ p_z^0 \end{pmatrix} = \begin{pmatrix} \hat{i}^0 \cdot \hat{i}^1 & \hat{i}^0 \cdot \hat{j}^1 & \hat{i}^0 \cdot \hat{k}^1 \\ \hat{j}^0 \cdot \hat{i}^1 & \hat{j}^0 \cdot \hat{j}^1 & \hat{j}^0 \cdot \hat{k}^1 \\ \hat{k}^0 \cdot \hat{i}^1 & \hat{k}^0 \cdot \hat{j}^1 & \hat{k}^0 \cdot \hat{k}^1 \end{pmatrix} \begin{pmatrix} p_x^1 \\ p_y^1 \\ p_z^1 \end{pmatrix}.$$

From the geometry of Figure 2-2 we could also write this same matrix as

$$\mathbf{p}^0 = \mathcal{R}_z(\theta)\mathbf{p}^1 = \mathcal{R}_1^0(\theta)\mathbf{p}^1 \tag{2.1}$$

where

$$\mathcal{R}_z(\psi) \triangleq \begin{pmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{pmatrix}. \tag{2.2}$$

The notation $\mathcal{R}_z$ is used to denote a physical rotation from coordinate frame $\mathcal{F}^0$ to coordinate frame $\mathcal{F}^1$, where the rotation is about the z axis of $\mathcal{F}^0$. However, this same matrix will also convert a vector in frame 1 to frame 0, or express it in the 0^{th} coordinate frame.

Proceeding in a similar way, a right-handed rotation of the coordinate system about the y-axis of $\mathcal{F}^0$ by an angle of θ gives

$$\mathcal{R}_y(\theta) \triangleq \begin{pmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{pmatrix}, \tag{2.3}$$

and a right-handed rotation of the coordinate system about the x-axis of $\mathcal{F}^0$ by an angle of ϕ is

$$\mathcal{R}_x(\phi) \triangleq \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{pmatrix}. \tag{2.4}$$

TOC

The matrices $\mathcal{R}_*$ in the above equations are examples of a more general class of *orthonormal* rotation matrices that have the following properties:

P.1. $(\mathcal{R}_a^b)^{-1} = (\mathcal{R}_a^b)^\top = \mathcal{R}_b^a$.

P.2. $\mathcal{R}_b^c \mathcal{R}_a^b = \mathcal{R}_a^c$.

P.3. $\det\left(\mathcal{R}_a^b\right) = 1$,

where $\det(\cdot)$ is the determinant of a matrix.

There are several coordinate frames associated with the hummingbird. The rotation between these frames can be described using rotation matrices as explained below. The world frame $\mathcal{F}^w$ is the coordinate frame that is centered at the center of rotation of the hummingbird. When the roll, pitch, and yaw angles are all zero, the $\hat{\imath}^w$-axis is aligned along the body and shaft of the hummingbird, the $\hat{\jmath}^w$-axis is to the right, and the $\hat{k}^w$-axis points down, as shown in Figure 2-3.

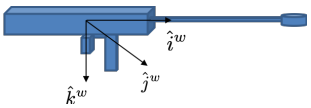


Figure 2-3: The hummingbird world frame. The $\hat{\imath}^w$-axis points along the roll axis, the $\hat{\jmath}^w$-axis points to the right, and the $\hat{k}^w$-axis points into the earth.

The hummingbird 1-frame, denoted $\mathcal{F}^1$, is obtained by rotating the $\mathcal{F}^w$ about the $\hat{k}^w$ axis by an angle of ψ, which is called the yaw angle. A top view of this rotation is shown in Figure 2-4. The rotation matrix that transforms coordinates

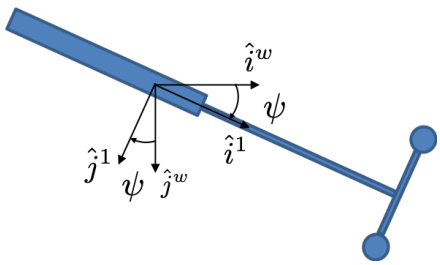


Figure 2-4: The hummingbird 1-frame is obtained from the world frame by rotating ψ about $\hat{k}^w$.

from $\mathbf{F}^w$ to $\mathbf{F}^1$ is $R_z(\psi)$ as given in Equation (2.2).

The hummingbird 2-frame, denoted $\mathcal{F}^2$, is obtained by rotating $\mathcal{F}^1$ about the $\hat{\jmath}^1$-axis by an angle of θ, which is called the pitch angle. A side view of this rotation is shown in Figure 2-5. The rotation matrix that transforms coordinates from $\mathbf{F}^1$ to $\mathbf{F}^2$ is $R_y(\theta)$ as given in Equation (2.3).

The hummingbird body frame, denoted $\mathcal{F}^b$, is obtained by rotating $\mathcal{F}^2$ about the $\hat{\imath}^2$-axis by an angle of ϕ, which is called the roll angle. A view of the head

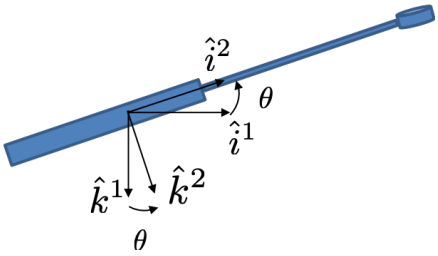


Figure 2-5: The hummingbird 2-frame is obtained from the hummingbird 1-frame by rotating θ about $\hat{j}^1$.

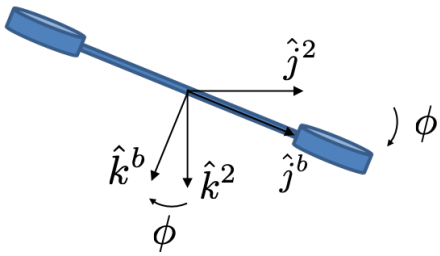


Figure 2-6: The hummingbird body frame is obtained from the hummingbird 2-frame by rotating ϕ about $\hat{i}^2$.

looking from the rotation center along the hummingbird shown in Figure 2-6. The rotation matrix that transforms coordinates from $\mathbf{F}^2$ to $\mathbf{F}^b$ is $R_x(\phi)$ as given in Equation (2.4). When performing sequential rotations, these rotation matrices can be composed (or multiplied) to find the total rotation. This will be shown in the next section.

2.2.1 Kinetic energy of the hummingbird

The hummingbird spins about the world z^w axis by the yaw angle ψ as shown in Figure 2-1. After rotating about z^w by ψ, the hummingbird spins about the body y^b-axis by the pitch angle θ. After yawing and pitching, the hummingbird spins about the body x^b-axis by the roll angle ϕ, as shown in Figure 2-1. Therefore, the configuration of the hummingbird is given by the three generalized variables (along with other known geometric quantities such as length of the rods, etc.)

$$\mathbf{q} = \begin{pmatrix} \phi \\ \theta \\ \psi \end{pmatrix}.$$

The position of the center of mass of m_3 is a function of the yaw angle ψ. The position of the center of mass of m_2 is a function of both the yaw angle ψ and the pitch angle θ, and the position of the center of mass of m_1 is a function of all three

angles ψ, θ, and ϕ. Therefore, the total kinetic energy of the system is given by

$$K = \sum_{i=1}^{3} \left(\frac{1}{2} m_i \mathbf{v}_i^T \mathbf{v}_i + \frac{1}{2} \boldsymbol{\omega}_i^T R_i J_i R_i^T \boldsymbol{\omega}_i \right).$$

where $\mathbf{v_i}$, $\boldsymbol{\omega}_i$, and R_i are functions of $\mathbf{q}$ and $\dot{\mathbf{q}}$. It turns out that $\mathbf{v_i}$ and $\boldsymbol{\omega}_i$ are linear in $\dot{\mathbf{q}}$. Therefore it is also possible to write the exact same expression as follows:

$$K = \sum_{i=1}^{3} \left(\frac{1}{2} m_i \dot{\mathbf{q}}^\top V_i^\top V_i \dot{\mathbf{q}} + \frac{1}{2} \dot{\mathbf{q}}^\top W_i^\top R_i J_i R_i^T W_i \dot{\mathbf{q}} \right) \tag{2.5}$$

$$= \frac{1}{2} \dot{\mathbf{q}}^\top \left(\sum_{i=1}^{3} m_i V_i^\top V_i + W_i^\top R_i J_i R_i^T W_i \right) \dot{\mathbf{q}} \tag{2.6}$$

$$\triangleq \frac{1}{2} \dot{\mathbf{q}}^\top M(\mathbf{q}) \dot{\mathbf{q}}, \tag{2.7}$$

where

$$M(\mathbf{q}) = \sum_{i=1}^{3} m_i V_i^\top(\mathbf{q}) V_i(\mathbf{q}) + W_i^\top(\mathbf{q}) R_i J_i R_i^T W_i(\mathbf{q}). \tag{2.8}$$

and V_i and W_i are matrices that are three by the length of the vector q.

This inner matrix $M(\mathbf{q})$ is called the generalized mass matrix and is important for a number of reasons. **The main goal of this chapter is to derive symbolic expressions for this matrix** (and therefore for the kinetic energy according to Eq. 2.7).

If the position of the center of mass of m_1 in the hummingbird 2-frame is given by $\mathbf{p}_1^b = \begin{pmatrix} \ell_1 & 0 & 0 \end{pmatrix}^\top$, (meaning it is a point on body 1, but expressed in frame b) then we can describe that same vector in the world frame by using appropriate rotation matrices given by

$$\mathbf{p}_1^w = R_z(\psi) R_y(\theta) R_x(\phi) \mathbf{p}_1^b$$

Using SymPy, show that the result for this is the following:

$$\mathbf{p}_1^w = \begin{pmatrix} \ell_1 c_\theta c_\psi \\ \ell_1 c_\theta s_\psi \\ -\ell_1 s_\theta \end{pmatrix}, \tag{2.9}$$

where to make the expressions easier to read, we have used the notation $c_\phi \triangleq cos(\phi)$ and $s_\phi \triangleq sin(\phi)$. Although it may have been obvious due to the geometry, because of where the center of mass is for body 1, the ϕ variable does not show up in Eqn. 2.9.

Hint: you may have to use the SymPy "simplify" function to see it in the same form as above, and you can use the `sym_utils.py` file in the repository

(`src/case_studies/common/`) to provide rotx, roty, rotz functions to calculate R_x, R_y, and R_z.

Now find the derivative with respect to time of $\mathbf{p}_1^w$ and show that the velocity of the center of mass for body 1 is equal to the following:

$$\mathbf{v}_1 = \dot{\mathbf{p}}_1 = \begin{pmatrix} -\ell_1\dot{\theta}s_\theta c_\psi - \ell_1\dot{\psi}c_\theta s_\psi \\ -\ell_1\dot{\theta}s_\theta s_\psi + \ell_1\dot{\psi}c_\theta c_\psi \\ -\ell_1\dot{\theta}c_\theta \end{pmatrix}. \tag{2.10}$$

We can use the "find_coeffs" function in `sym_utils.py` to now factor out $\dot{\mathbf{q}}$ from our $\mathbf{v}_1$ vector. Verify that the result is the following:

$$V_1(\mathbf{q})\dot{\mathbf{q}} \triangleq \begin{pmatrix} 0 & -\ell_1 s_\theta c_\psi & -\ell_1 c_\theta s_\psi \\ 0 & -\ell_1 s_\theta s_\psi & \ell_1 c_\theta c_\psi \\ 0 & -\ell_1 c_\theta & 0 \end{pmatrix} \begin{pmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{pmatrix}. \tag{2.11}$$

Hopefully you can see where we are going with this process. We will next find the positions and velocities for bodies 2 and 3.

If the position of the center of mass of m_2 in the body frame is given by $\mathbf{p}_2^2 = \begin{pmatrix} \ell_2 & 0 & 0 \end{pmatrix}^\top$, then in the inertial frame, the position is given by

$$\mathbf{p}_2^w = R_z(\psi)R_y(\theta) \begin{pmatrix} \ell_2 \\ 0 \\ 0 \end{pmatrix}$$

Find the velocity of the center of mass for the second rigid body, and show that it is equal to the following (where to make the expressions easier to read, we have used the notation $c_\phi \triangleq \cos(\phi)$ and $s_\phi \triangleq \sin(\phi)$):

$$\mathbf{v}_2 = \dot{\mathbf{p}}_2 = \begin{pmatrix} -\ell_2\dot{\theta}s_\theta c_\psi - \ell_2\dot{\psi}c_\theta s_\psi \\ -\ell_2\dot{\theta}s_\theta s_\psi + \ell_2\dot{\psi}c_\theta c_\psi \\ -\ell_2\dot{\theta}c_\theta \end{pmatrix}. \tag{2.12}$$

Again, using the "find_coeffs" function, find V_2 as shown below:

$$V_2(\mathbf{q})\dot{\mathbf{q}} \triangleq \begin{pmatrix} 0 & -\ell_2 s_\theta c_\psi & -\ell_2 c_\theta s_\psi \\ 0 & -\ell_2 s_\theta s_\psi & \ell_2 c_\theta c_\psi \\ 0 & -\ell_2 c_\theta & 0 \end{pmatrix} \begin{pmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{pmatrix}. \tag{2.13}$$

Finally, if the position of the center of mass of m_3 in frame 1 is given by $\mathbf{p}_3^1 = \begin{pmatrix} \ell_{3x} & \ell_{3y} & \ell_{3z} \end{pmatrix}^\top$, then in the inertial frame, the position is given by

$$\mathbf{p}_3^w = R_z(\psi) \begin{pmatrix} \ell_{3x} \\ \ell_{3y} \\ \ell_{3z} \end{pmatrix}$$

Therefore, show that the velocity of the third mass is given by

$$\mathbf{v}_3 = \dot{\mathbf{p}}_3 = \dot{\psi} \begin{pmatrix} -\ell_{3x} s_\psi - \ell_{3y} c_\psi \\ \ell_{3x} c_\psi - \ell_{3y} s_\psi \\ 0 \end{pmatrix}, \tag{2.14}$$

and that V_3 is defined as follows:

$$\mathbf{v}_3 = V_3(\mathbf{q})\dot{\mathbf{q}} \tag{2.15}$$

$$= \begin{pmatrix} 0 & 0 & -(\ell_{3x} s_\psi + \ell_{3y} c_\psi) \\ 0 & 0 & (\ell_{3x} c_\psi - \ell_{3y} s_\psi) \\ 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{pmatrix}. \tag{2.16}$$

Based on Eqns. 2.6 and 2.7, we have half of the terms we need. However, calculating the angular velocity of each rigid body can be a little trickier. Without significant background, we will introduce a useful method for calculating $\boldsymbol{\omega}_i$ given that we know the orientation or rotation of body i relative to the inertial frame. It turns out that if we can write the orientation of body i as a rotation matrix which is a function of our generalized coordinates, then the following is true:

$$\Omega = \begin{pmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{pmatrix} = \dot{R}_i R_i^T \tag{2.17}$$

where $\dot{R}_i$ is simply the element-wise time derivative of the rotation matrix and R_i^T is simply the transpose of the rotation matrix. We have implemented this function for you in the "calc_omega" function in `sym_utils.py`. To use this function correctly, we will need to define rotation matrices for each rigid body, calculate omega $\boldsymbol{\omega}_i$, and then use the "find_coeffs" function to find W_i (similarly to what we did for the linear velocities).

This method of finding $\boldsymbol{\omega}$ is straightforward, but it is calculating the angular velocity in a global or inertial coordinate frame. In order to combine it with the other terms in Eqn. 2.6, we will need to rotate the inertia tensor (which is expressed in the body frame) into the inertial frame as well. This operation is shown in Eqn. 2.6 where J_i is pre- and post-multiplied by R_i and $R_i.T$ transpose respectively.

The first step is to find R_i for each rigid body. For body one, this simply means defining the body frame "b", relative to the world frame as follows:

$$R_1 = R_z(\psi) R_y(\theta) R_x(\phi).$$

Similarly, we can find the rotations for body 2 and 3 as well:

$$R_2 = R_z(\psi) R_y(\theta)$$
$$R_3 = R_z(\psi)$$

If you noticed that these are the same as the rotations that we used to transform the center of mass of each body into the inertial frame, you are correct! We will use these to find $\boldsymbol{\omega}_i$ *AND* to rotate the inertia tensor as shown in Eqn. 2.6.

We provide both the expressions for $\boldsymbol{\omega}_i$ and W_i below for you to verify the equations that you will find in code:

$$\boldsymbol{\omega}_1 = \begin{pmatrix} -s_\psi\dot{\theta} + c_\psi c_\theta\dot{\phi} \\ s_\psi c_\theta\dot{\phi} + c_\psi\dot{\theta} \\ -s_\theta\dot{\phi} + \dot{\psi} \end{pmatrix} \tag{2.18}$$

$$W_1 = \begin{pmatrix} c_\psi c_\theta & -s_\psi & 0 \\ s_\psi c_\theta & c_\psi & 0 \\ -s_\theta & 0 & 1 \end{pmatrix} \tag{2.19}$$

$$\boldsymbol{\omega}_2 = \begin{pmatrix} -s_\psi\dot{\theta} \\ c_\psi\dot{\theta} \\ \dot{\psi} \end{pmatrix} \tag{2.20}$$

$$W_2 = \begin{pmatrix} 0 & -s_\psi & 0 \\ 0 & c_\psi & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{2.21}$$

$$\boldsymbol{\omega}_3 = \begin{pmatrix} 0 \\ 0 \\ \dot{\psi} \end{pmatrix} \tag{2.22}$$

$$W_3 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{2.23}$$

Now we can use the terms that we have found to calculate $M(q)$ according to Eqn. 2.8. Do this in your code and make sure to verify that your result looks like the following:

$$M(\mathbf{q}) = \begin{pmatrix} J_{1x} & 0 & -J_{1x}s_\theta \\ 0 & M_{22} & M_{23} \\ -J_{1x}s_\theta & M_{23} & M_{33} \end{pmatrix} \tag{2.24}$$

where

$$M_{22} = J_{1y}c_\phi^2 + J_{1z}s_\phi^2 + J_{2y} + m_1\ell_1^2 + m_2\ell_2^2 \tag{2.25}$$

$$M_{23} = (J_{1y} - J_{1z})s_\phi c_\phi c_\theta \tag{2.26}$$

$$M_{33} = J_{3z} + \ell_{3x}^2 m_3 + \ell_{3y}^2 m_3 + (J_{1x} + J_{2x})\, s_\theta^2 + \tag{2.27}$$

$$\left(J_{1y}s_\phi^2 + J_{1z}c_\phi^2 + J_{2z} + \ell_1^2 m_1 + \ell_2^2 m_2\right) c_\theta^2 \tag{2.28}$$

$$\tag{2.29}$$

As you pass this lab off to the TA, be sure to discuss what the $M(\mathbf{q})$ matrix means, and what insight we might gain by examining its role in Eqn. 2.7.

TOC

Lab Assignment H.2

1. Read through this chapter and find each of the symbolic expressions required (marked with a “TODO" in the `generate_KE.py` file (`src/case_studies/H_hummingbird/` in the course repository)). For now, all you need is the mass matrix $M(\mathbf{q})$, but by finding this, it means that you can easily find the kinetic energy from Eqn. 2.7.

2. Fill out the parameters in `params.py` (`src/case_studies/H_hummingbird/`) from Appendix a.

3. Finally, write a short driver script (`labs/h2_animate.py`), similar to the animation homework for case studies D, E, and F, that uses the provided visualizer, and run it varying each configuration variable (i.e. roll, pitch, yaw), one at a time, and make sure you understand the sign convention. How does the hummingbird move when the roll ϕ is positive? Similarly for pitch θ, and yaw ψ.

Hints

To run the animation, you will need the required python packages. Please make sure to follow the public repository installation instructions which use the pyproject.toml file to install the required libraries in a virtual environment.

3 The Euler Lagrange Equations

The objective of this Chapter is to define the potential energy for the hummingbird, and show how to use the hummingbird's kinetic and potential energy to derive the equations of motion using the Euler-Lagrange equation.

3.1 Potential Energy

In this section, we will define the hummingbird's potential energy.

Since there are no springs in the system, the only source of potential energy is due to gravity. Therefore the potential energy is given by

$$P = m_1 g h_1 + m_2 g h_2 + m_3 g h_3$$

where h_1 is the height of the center of mass of m_1 and h_2 is the height of the center of m_2, etc. From the geometry of the hummingbird we have

$$P(q) = m_1 g \ell_1 s_\theta + m_2 g \ell_2 s_\theta + m_3 g \ell_{3z} + P_0$$

It is also acceptable to group $m_3 g \ell_{3z}$ with P_0 since it is a constant and not a function of $\mathbf{q}$.

3.2 Euler-Lagrange Equations

The objective of this section is to develop the simulation model for the hummingbird using the Euler-Lagrange equation

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\mathbf{q}}}\right) - \frac{\partial L}{\partial \mathbf{q}} = \boldsymbol{\tau} - B\dot{\mathbf{q}}, \tag{3.1}$$

where L is the Lagrangian, $\mathbf{q}$ is the vector of generalized coordinates, $\boldsymbol{\tau}$ are the generalized forces, and B is the damping matrix.

TOC

The Lagrangian is defined as

$$\begin{aligned} L &\triangleq K(\mathbf{q}, \dot{\mathbf{q}}) - P(\mathbf{q}) \\ &= \frac{1}{2}\dot{\mathbf{q}}^T M(\mathbf{q})\dot{\mathbf{q}} - P(\mathbf{q}). \end{aligned}$$

We found the kinetic energy previously in Chap. 2, and the potential energy is defined in Sec. 3.1. The generalized forces are given as follows:

$$\boldsymbol{\tau} = \begin{pmatrix} d(f_\ell - f_r) \\ \ell_T(f_\ell + f_r)c_\phi \\ \ell_T(f_\ell + f_r)c_\theta s_\phi - d(f_\ell - f_r)s_\theta \end{pmatrix}, \tag{3.2}$$

where ℓ_T is the length from the pivot point to the head of the hummingbird.

For the following derivations of each term, it is expected that you use SymPy tools to validate each term and then incorporate it into your code. You will need to show the output of your code to a TA for passing off the lab. You can then, of course, type these terms directly into your code, but that is error prone and much more difficult. It is recommended that you use the "write_eom_to_file" function instead to generate numpy-based functions for use in your dynamics file.

For the first term in Eqn. 3.1, we can write it as follows:

$$\frac{d}{dt}\frac{\partial L}{\partial \dot{\mathbf{q}}} = \frac{d}{dt}M(\mathbf{q})\dot{\mathbf{q}} = M(\mathbf{q})\ddot{\mathbf{q}} + \dot{M}\dot{\mathbf{q}}. \tag{3.3}$$

For the second term in Eqn. 3.1, taking the partial of L with respect to $\mathbf{q}$ gives

$$\frac{\partial L}{\partial \mathbf{q}} = \frac{1}{2}\begin{pmatrix} \dot{\mathbf{q}}^T \frac{\partial M}{\partial q_1} \\ \dot{\mathbf{q}}^T \frac{\partial M}{\partial q_2} \\ \dot{\mathbf{q}}^T \frac{\partial M}{\partial q_3} \end{pmatrix} \dot{\mathbf{q}} - \frac{\partial P}{\partial \mathbf{q}}. \tag{3.4}$$

Where $\mathbf{q}$ is assumed to be a column vector, and each $\dot{\mathbf{q}}^T \frac{\partial M}{\partial q_i}$ term is a $1 \times n$ vector.

Therefore, the Euler-Lagrange equation gives

$$M(\mathbf{q})\ddot{\mathbf{q}} + \left[\dot{M} - \frac{1}{2}\begin{pmatrix} \dot{\mathbf{q}}^T \frac{\partial M}{\partial q_1} \\ \dot{\mathbf{q}}^T \frac{\partial M}{\partial q_2} \\ \dot{\mathbf{q}}^T \frac{\partial M}{\partial q_3} \end{pmatrix}\right] \dot{\mathbf{q}} + \frac{\partial P}{\partial \mathbf{q}} = \boldsymbol{\tau} - B\dot{\mathbf{q}}. \tag{3.5}$$

If we define

$$C(\mathbf{q}, \dot{\mathbf{q}}) \triangleq \dot{M}\dot{\mathbf{q}} - \frac{1}{2}\begin{pmatrix} \dot{\mathbf{q}}^T \frac{\partial M}{\partial q_1} \\ \dot{\mathbf{q}}^T \frac{\partial M}{\partial q_2} \\ \dot{\mathbf{q}}^T \frac{\partial M}{\partial q_3} \end{pmatrix} \dot{\mathbf{q}}, \tag{3.6}$$

then the equations of motion are given by

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}}) + \frac{\partial P}{\partial \mathbf{q}} = \boldsymbol{\tau} - B\dot{\mathbf{q}}. \tag{3.7}$$

TOC

Taking the time derivative of M, we can find $\dot{M}$ is the following:

$$\dot{M} = \begin{pmatrix} 0 & 0 & -J_{1x}c_\theta\dot{\theta} \\ 0 & \dot{M}_{22} & \dot{M}_{23} \\ -J_{1x}c_\theta\dot{\theta} & \dot{M}_{23} & \dot{M}_{33} \end{pmatrix}, \tag{3.8}$$

where

$$\dot{M}_{22} = 2(J_{1z} - J_{1y})s_\phi c_\phi \dot{\phi} \tag{3.9}$$

$$\dot{M}_{23} = (J_{1y} - J_{1z})\left(-2s_\phi^2 c_\theta\dot{\phi} - s_\phi s_\theta c_\phi\dot{\theta} + c_\theta\dot{\phi}\right) \tag{3.10}$$

$$\dot{M}_{33} = -2\big((-J_{1y} + J_{1z})s_\phi c_\phi c_\theta\dot{\phi} \tag{3.11}$$

$$+ (-J_{1x} + J_{1y}s_\phi^2 + J_{1z}c_\phi^2 - J_{2x} \tag{3.12}$$

$$+ J_{2z} + \ell_1^2 m_1 + \ell_2^2 m_2)\dot{\theta}s_\theta\big)c_\theta.$$

In addition, we can calculate the partial derivatives of M as follows:

$$\frac{\partial M}{\partial \phi} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & (J_{1z} - J_{1y})s_{(2\phi)} & (J_{1y} - J_{1z})c_{(2\phi)}c_\theta \\ 0 & (J_{1y} - J_{1z})c_{(2\phi)}c_\theta & 2(J_{1y} - J_{1z})s_\phi c_\phi c_\theta^2 \end{pmatrix} \tag{3.13}$$

$$\frac{\partial M}{\partial \theta} = \begin{pmatrix} 0 & 0 & -J_{1x}c_\theta \\ 0 & 0 & -(J_{1y} - J_{1z})s_\phi c_\phi s_\theta \\ -J_{1x}c_\theta & -(J_{1y} - J_{1z})s_\phi c_\phi s_\theta & N_{33} \end{pmatrix} \tag{3.14}$$

$$\frac{\partial M}{\partial \psi} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \tag{3.15}$$

where

$$N_{33} = 2\left(J_{1x} - J_{1y}s_\phi^2 - J_{1z}c_\phi^2 + J_{2x} - J_{2z} - \ell_1^2 m_1 - \ell_2^2 m_2\right)s_\theta c_\theta.$$

We can now combine these terms to calculate $C(\mathbf{q}, \dot{\mathbf{q}})$, as well as calculating $\frac{\partial P}{\partial \mathbf{q}}$ in your code.

We will model damping in each axis of rotation (or in the physical bearings) using the viscous friction model $-B\dot{\mathbf{q}}$, with

$$B = \beta I, \tag{3.16}$$

and $\beta = 0.001$.

3.2.1 Summary

The equations of motion are

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}}) + \frac{\partial P}{\partial \mathbf{q}} = \boldsymbol{\tau} - B\dot{\mathbf{q}}, \tag{3.17}$$

TOC

where

$$\mathbf{q} = \begin{pmatrix} \phi \\ \theta \\ \psi \end{pmatrix},$$

are the roll, pitch, and yaw angles.

Debugging these terms is difficult, but we have provided test cases in the repository that will allow you to check your M, C, $\frac{\partial P}{\partial \mathbf{q}}$, τ and friction terms. Please do this so that you can be sure your code is likely to work.

Lab Assignment H.3

1. Read through this chapter and find symbolic versions of the equations of motion in Equation (3.17) (i.e. M, C, $\frac{\partial P}{\partial \mathbf{q}}$, and τ) using SymPy or similar tool. Complete the marked "TODO" sections in `generate_state_variable_form.py` (`src/case_studies/H_hummingbird/`) to do this. Write numpy functions to calculate the resultant terms required in Equation (3.17) and verify them with the provided test cases. Alternatively, you may copy or re-write these symbolic terms as Python code manually, but this is likely to be difficult to debug and error-prone. Show the TA that your expressions match the terms in the lab manual.

2. Copy `dynamics_template.py` to `dynamics.py` in `src/case_studies/H_hummingbird/` and complete the "TODO" sections to implement the equations of motion for the hummingbird, i.e. calculate $f(x, u)$. Use the file `testDynamics.py` to verify that the dynamics are implemented correctly. Show the TA that your values are correct.

3. Create a file `labs/h3_sim.py` that simulates the system for various forces on the input. The simulation may not do anything meaningful, i.e., it may just swing around wildly. However, checking your dynamics now using tools provided by the TAs and professor may save you hours of time later. One simple way to test your dynamics is to do something like applying a constant force (f_l + f_r) that is equal to the torque induced about the pitch axis due to gravity. If you do this correctly, the hummingbird should not move at all. Any minor changes to f_l and f_r beyond this will cause it to roll and then yaw in the direction (right or left) that is applying less force. These types of tests are very valuable when first implementing dynamics and testing them in a simulation. Show your simulation to a TA.

Hints

- Just go slowly and make sure to double check every entry (including intermediate steps like the M, C, τ, and friction terms).

- You should expect that both forces ($f_l = f_r$) should be small to get reasonable behavior ($\approx \frac{0.23}{2.0}$) for initial testing.

Part II

Hummingbird Design Models

TOC

4

Equilibria and Linearization

The equations of motion developed in the previous chapter may be too complex to be useful for control design (especially in the case of PID or transfer function-based control design). Therefore the objective of this chapter is to develop several design models for the hummingbird that capture certain behaviors of the system.

The essential idea is to notice that since the total force on the end of the hummingbird is $F = f_l + f_r$, the pitching motion can be controlled by manipulating F (and treating it as one input). Also, since the torque on the end of the hummingbird is $\tau = (f_l - f_r)d$, the rolling motion can be controlled by manipulating τ (and treating it as another input). We also note that the yawing motion is driven primarily by the roll angle ϕ.

Accordingly, we define the *longitudinal* motion of the hummingbird, to be the motion involving the state variables related to pitch $(\theta, \dot{\theta})$, and the *lateral* motion of the hummingbird to be the motion involving the state variables for roll and yaw $(\phi, \psi, \dot{\phi}, \dot{\psi})$. While these motions are coupled, the coupling is fairly weak and will be neglected in the design models.

In this chapter we will discuss finding the equilibria and linearization of the longitudinal and lateral dynamics of the hummingbird.

4.1 Forming Equations of Motion with Total Force and Torque

For the first step in this lab, we will replace the control inputs of thrust for each motor (f_l and f_r) with the total force and torque (F and T).

There are likely many ways to do this, but to avoid complex substitutions and simplifications in SymPy, we will find an equation for f_l and f_r in terms of F and T, then substitute them into our equations of motion directly.

Do this by using the SymPy "Eq" function which makes a symbolic equation

given a right and left hand side. More specifically, it would look something like this if we have symbolic variables f_l, f_r and F defined:

```
import sympy as sp
sp.Eq(f_l + f_r, F)
```

We can make an additional equation for torque (T) in terms of f_l and f_r, and then solve for f_l and f_r in terms of T and F using the `sp.solve` function. See notes in `generate_linearization.py` for more guidance.

Once you have found an expression for f_l and f_r, you can substitute them into the expression for the generalized forces (τ) from the previous lab using `tau.subs()`.

4.2 Formulate Full Symbolic Equations of Motion and Find Equilibria

After finding the new form for our generalized forces, we can once again use the `sp.Eq` function in SymPy to define our full equations of motion. We recommend that you do this by first defining a right hand side of the equation (RHS) and a left hand side of the equation (LHS), that can be combined as follows:

```
eom = sp.Eq(LHS, RHS)
```

After we have this "eom" SymPy variable, we can use it to help us find the equilibrium values for the system.

Using the guidance in the lab template, find the equilibria (x_0 and u_0) and store them for later use. This will require the following steps:

1. Defining equilibrium variables such as ϕ_e, ψ_e, and θ_e. As well as an equilibrium force (F_e) and torque (T_e).

2. Defining nonsymbolic values at equilibrium (such as $\dot{\phi} = \ddot{\phi} = 0$).

3. Grouping all of these equilibrium point variables into a single Python dictionary, and then substituting them into the original equations of motion. This will only be used to find the equilibria of the system.

4. Following the reasoning in the lab template to find the remaining equilibrium variable values.

We will store the necessary equilibrium values (especially F_e since all other values end up being zero!). It is important to follow the reasoning and derivation of these variables, even if they are mostly zero.

TOC

4.3 Linearization for the Longitudinal Dynamics

In this section, we will linearize the longitudinal dynamics derived in the previous chapter. One possible approach is to find the linearization of both the longitudinal and lateral dynamics simultaneously as shown in the course textbook (where $\dot{x} = f(x,u)$ and we find $A = \frac{\partial f}{\partial x}|_{x=x_0,u=u_0}$ and $B = \frac{\partial f}{\partial u}|_{x=x_0,u=u_0}$). This process is effective, especially if using a symbolic manipulation tool like SymPy. However, it may result in equations that are more difficult to interpret or use directly in PID control design.

Instead, since we are only interested in the longitudinal dynamics, we will use the governing equation for θ stated in the second row of Equation (3.17) which is

$$\begin{aligned}
&\frac{1}{2}J_{1x}c_\theta\dot{\phi}\dot{\psi} + gc_\theta\left(\ell_1 m_1 + \ell_2 m_2\right) + \frac{1}{2}\left(J_{1y} - J_{1z}\right)s_\phi s_\theta c_\phi \dot{\psi}\dot{\theta} + \\
&\left(J_{1y} - J_{1z}\right)s_\phi c_\phi c_\theta \ddot{\psi} + 2\left(-J_{1y} + J_{1z}\right)s_\phi c_\phi \dot{\phi}\dot{\theta} - \\
&\left[-\frac{1}{2}J_{1x}c_\theta\dot{\phi} - \frac{1}{2}\left(J_{1y} - J_{1z}\right)s_\phi s_\theta c_\phi \dot{\theta} + \right. \\
&\left.\left(J_{1x} + J_{1y}c_\phi^2 - J_{1y} - J_{1z}c_\phi^2 + J_{2x} - J_{2z} - \ell_1^2 m_1 - \ell_2^2 m_2\right)s_\theta c_\theta \dot{\psi}\right]\dot{\psi} + \\
&\left(-\left(J_{1y} - J_{1z}\right)s_\phi^2 c_\theta \dot{\phi} - \left(J_{1y} - J_{1z}\right)s_\phi s_\theta c_\phi \dot{\theta} + \left(J_{1y} - J_{1z}\right)c_\phi^2 c_\theta \dot{\phi}\right)\dot{\psi} \\
&+ \left(-J_{1y}s_\phi^2 + J_{1y} + J_{1z}s_\phi^2 + J_{2y} + \ell_1^2 m_1 + \ell_2^2 m_2\right)\ddot{\theta} \\
&= -\beta\dot{\theta} + \ell_T\left(f_l + f_r\right)c_\phi \qquad (4.1)
\end{aligned}$$

In order to find our linearized longitudinal dynamics, we can do the following steps after finding the equation above:

1. Set $\phi_e = \dot{\phi} = \psi_e = \dot{\psi} = \ddot{\psi} = 0$ in your equations (essentially assuming that the lateral dynamics have no effect on the longitudinal dynamics at equilibrium). We can do this using the "subs" function in SymPy.

2. Solve for a feedback linearization force ($F = F_{fl} + F_{control}$) that cancels the remaining $gcos(\theta)$ term and makes this equation linear in the state and input. It may be helpful to examine the equation of motion after doing the substitution in the previous step. Plugging this result for F_{fl} back into that equation should give the following (verify this in your code):

$$\ddot{\theta} + \left(\frac{\beta}{m_1\ell_1^2 + m_2\ell_2^2 + J_{1y} + J_{2y}}\right)\dot{\theta} = \left(\frac{\ell_T}{m_1\ell_1^2 + m_2\ell_2^2 + J_{1y} + J_{2y}}\right)F_{control}. \qquad (4.2)$$

3. Finally, set the friction coefficient $b = 0$.

Neglecting friction may seem like an odd choice, but it helps for multiple reasons. First, it simplifies our dynamics, second, b is often difficult to identify

experimentally, and third if there is friction on the real system, assuming there is none in our model is generally a conservative assumption for developing controllers. It is important to note that for the hummingbird, we will perform feedback linearization for the longitudinal dynamics, but a Jacobian linearization for the lateral dynamics. This is driven by a desire to improve performance where feedback linearization is possible for the θ state.

The result of all of these steps should be the following:

$$\ddot{\theta} = \left(\frac{\ell_T}{m_1 \ell_1^2 + m_2 \ell_2^2 + J_{1y} + J_{2y}} \right) F_{control} = b_\theta \tilde{F}, \tag{4.3}$$

where

$$b_\theta = \left(\frac{\ell_T}{m_1 \ell_1^2 + m_2 \ell_2^2 + J_{1y} + J_{2y}} \right). \tag{4.4}$$

In Eqn. 4.3, we have substituted $F_{control}$ for $\tilde{F}$. This is not strictly necessary, especially since we did not perform Jacobian linearization. However, it is convenient to treat all variables as being referenced to an equilibrium (including the longitudinal states and the force). This notation will make our equations congruent with the lateral dynamics that require linearization and cannot be accomplished with feedback linearization.

4.4 Linearization for the Lateral Dynamics

The objective of this section is to linearize the lateral dynamics. The governing equations for ϕ and ψ are the first and third lines of Equation (3.17). Following the steps below, we can linearize these equations of motion:

1. Set $\theta = \dot{\theta} = \ddot{\theta} = 0$ in your lateral dynamics equations. You can also set the friction to zero again ($\beta = 0$). This should result in the following equations:

$$J_{1x}\ddot{\phi} + \frac{1}{2}\left(-J_{1y} + J_{1z}\right) s_{2\phi} \dot{\psi}^2 = \mathrm{T} \tag{4.5}$$

$$\left(J_{1y} - J_{1z}\right) s_{2\phi} \dot{\phi}\dot{\psi} + \left(J_T\right) \ddot{\psi} = F \ell_T s_\phi \tag{4.6}$$

where

$$J_T \triangleq -J_{1y} c_\phi^2 + J_{1y} + J_{1z} c_\phi^2 + J_{2z} + J_{3z} + \ell_1^2 m_1 + \ell_2^2 m_2 + \ell_{3x}^2 m_3 + \ell_{3y}^2 m_3$$

2. The equilibrium force (F_e) that we found earlier is very similar to the feedback linearization force (F_{fl}), but a function of $\theta_e = 0$ instead of a function of $\theta(t)$. We are assuming that F does not affect the variables of interest (ϕ and ψ), but F_e still shows up as a constant so we need to use it.

3. Substitute the symbol F_e for F in your equations.

4. Now take the partials of each equation with respect to $x_{lat} = [\psi, \phi, \dot{\psi}, \dot{\phi}]^T$ and $u_{lat} = [T]$. Don't forget to substitute values for ϕ_e and T_e.

If you take the partial derivative matrices found in step 4, and realizing that we have now found tilde variables by linearizing, you should find the following linearized equations:

$$\ddot{\tilde{\phi}} = \frac{1}{J_{1x}}\tilde{\tau} \tag{4.7}$$

$$\ddot{\tilde{\psi}} = \frac{F_e \ell_T \tilde{\phi}}{J_{1z} + J_{2z} + J_{3z} + \ell_1^2 m_1 + \ell_2^2 m_2 + \ell_{3x}^2 m_3 + \ell_{3y}^2 m_3} \tag{4.8}$$

4.5 Model of the Motor-Propeller

In the previous sections, we modeled the motors as producing a force perpendicular to the hummingbird x-y plane. The input to the motors is actually a pulse-width-modulation (PWM) command that regulates the duty cycle of the current supplied to the motor. The motor stops when the PWM is below 1060 μs (0% duty cycle) and is at full power when the PWM is at 1860 μs (100% duty cycle). To make things easier, code has been added to the hummingbird's firmware to convert the duty cycle to PWM. That means that the output command to the hummingbird needs to be converted from force to duty cycle, and the duty cycle is constrained in the range $[0, 1]$, where 0 represents zero percent duty cycle, or zero current supplied to the motor, and 1 represents 100 percent duty cycle, or full current supplied to the motor.

While there are dynamics in the internal workings of the motor, we will neglect these dynamics and model the relationship between PWM command u and force as

$$F = f(u).$$

F is not linearly proportional to u, but rather a function of u. However, for the purposes of this class we will assume it to be linearly porportional to u by the scalar k_m.

$$F = k_m u.$$

We can make this assumption since F and u are almost linearly proportional to each other in the region of operation. The region of operation is centered around the force needed to maintain the hummingbird in equilibrium when $\psi = \theta = \phi = 0$.

To experimentally find the value of k_m we apply an equal PWM signal to each motor and increase this signal until the force balances the hummingbird at constant pitch angle. In the equilibrium position the forces cancel and we have

$$\ell_T F_e = m_1 \ell_1 g + m_2 \ell_2 g.$$

Setting $F_e = f_{l_e} + f_{r_e} = k_m(u_{l_e} + u_{r_e})$, and solving for k_m we get

$$k_m = \frac{m_1\ell_1 g + m_2\ell_2 g}{\ell_T(u_{l_e} + u_{r_e})}.$$

While PWM is the input to the system, it is easier to think in terms of torque and force applied to the hummingbird. To begin with, it is helpful to remember that the relationship between the PWM commands and the left (f_l) and right (f_r) forces is simply the following:

$$\begin{pmatrix} f_l \\ f_r \end{pmatrix} = k_m \begin{pmatrix} u_l \\ u_r \end{pmatrix}. \tag{4.9}$$

We can also relate u_l and u_r to the total force and torque acting on the hummingbird. This is given by the following:

$$\begin{pmatrix} \tau \\ F \end{pmatrix} = k_m \begin{pmatrix} d & -d \\ 1 & 1 \end{pmatrix} \begin{pmatrix} u_l \\ u_r \end{pmatrix}. \tag{4.10}$$

Inverting the matrix and solving for PWM command gives

$$u_l = \frac{1}{2k_m}\left(F + \frac{\tau}{d}\right) \tag{4.11}$$

$$u_r = \frac{1}{2k_m}\left(F - \frac{\tau}{d}\right). \tag{4.12}$$

You can use any of these relationships for the simulation, but you should update your update function (in `H_hummingbird/dynamics.py`) which should take in a vector of PWM values (u_l and u_r) and then convert them to either force (F) and torque (τ) or f_l and f_r (whichever is easier for the way you wrote your dynamics). The required equilibrium force (F_e) described below should then be converted into two PWM values (u_l and u_r) and sent as commands to the dynamics simulation.

Lab Assignment H.4

The objective of this lab is find the linearized equations using SymPy. We will then use our knowledge of the system to calculate the physical gain k_m that converts PWM level to force, where the force on the left and right motors are given by $f_\ell = k_m u_\ell$ and $f_r = k_m u_r$ and u_ℓ and u_r are between 0 and 1, where zero is no throttle, and one is full throttle.

1. Follow each step in the sections above, including the noted "TODO" sections in the `generate_linearization.py` template (`src/case_studies/H_hummingbird/`).

2. Implement the necessary changes to both your dynamics, and a new `labs/h4_sim.py` file to take in PWM and convert it to F and τ (or f_l and f_r if you prefer).

3. Next, use the simulator to verify that if k_m is known, then

$$\theta_e = 0, \qquad F_e = \frac{(m_1\ell_1 + m_2\ell_2)g}{\ell_T}$$

   is an equilibrium for the system. You can do this by setting the initial condition of the system to the equilibrium state, and then applying F_e and T_e as inputs in the simulation. If done correctly, the system should remain stationary.

4. Next, we will identify k_m experimentally on the hummingbird hardware. You may only proceed with hardware for the first time if a TA is in the lab. Follow the steps carefully for running the hummingbird in the "How to Use Hummingbird Hardware.PDF" file on the "Content
   Lab Material" page. Find the correct k_m value. You will follow these steps multiple times throughout the semester as you tune different controllers. So do your best to become familiar with how the hummingbird works.

5. Pass off the equations, the simulation, and the k_m value with a TA.

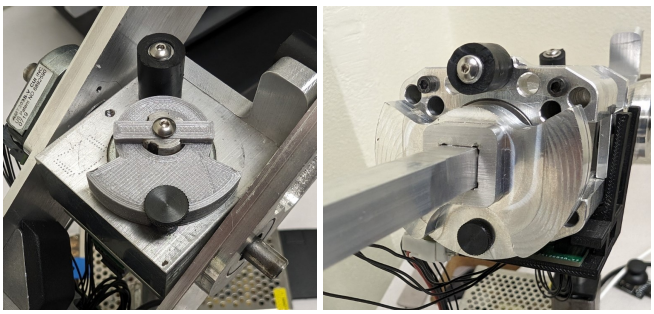


Figure 4-1: For this lab, we will only use the "pitch" degree of freedom (θ). Close-up images of the two screws used to restrict yaw (shown on left) and roll (shown on right) are shown. Make sure the screws are centered and well-seated.

5 Transfer Function Models

The response of a system to an input can be approximated or modeled by its transfer function. By modeling the linearized longitudinal and lateral dynamics with a transfer function, we can gain intuition on how the hummingbird will respond to a given force or torque input. These transfer functions will be used in a later chapter to design the control system for the hummingbird.

5.1 Linear Transfer Function Model for Longitudinal Dynamics

In the previous chapter, we decoupled (from the lateral generalized coordinates ϕ and ψ) and linearized the longitudinal equations of motion ($\ddot{\theta}$). The linearized equation (4.3) is a second-order differential equation relating $\tilde{\theta}$ and $\tilde{F}$. The output for the longitudinal transfer function is obtained by taking the Laplace transform of (4.3) and solving for $\tilde{\theta}(s)$. Find this transfer function.

5.2 Linear Transfer Function Model for Lateral Dynamics

We also decoupled the lateral equations of motion (for $\ddot{\psi}$ and $\ddot{\phi}$) from the longitudinal generalized coordinate θ and linearized them. The linearized equations are shown in Equation (4.8). The first equation is a second-order differential equation relating $\tilde{\tau}$ and $\ddot{\tilde{\phi}}$, while the second is a second-order differential equation relating $\tilde{\phi}$ and $\ddot{\tilde{\psi}}$.

Find the two relevant transfer functions for these equations.

The transfer functions we find will show that the lateral dynamics form a cascade structure as shown in Figure 5-1. As shown in Figure 5-1 the torque τ can be thought of as the input to the roll dynamics. While the roll angle ϕ which is the output of the roll dynamics can be thought of as the input to the yaw dynamics. This cascade structure will be utilized in later labs where we will design controllers for the roll dynamics and yaw dynamics separately and enforce suitable bandwidth separation (speed of response) to ensure a good design.

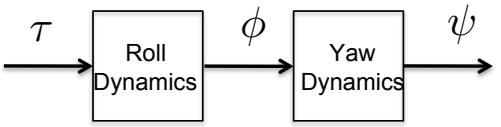


Figure 5-1: The lateral dynamics for the hummingbird can be thought of as a cascade system, where the torque drives the roll dynamics and the roll angle drives the yaw dynamics.

Lab Assignment H.5

1. Find the following transfer functions and pass them off to a TA:
   - $\frac{\tilde{\theta}(s)}{\tilde{F}(s)}$ - longitudinal dynamics
   - $\frac{\tilde{\phi}(s)}{\tilde{\tau}(s)}$ - lateral dynamics
   - $\frac{\tilde{\psi}(s)}{\tilde{\phi}(s)}$ - lateral dynamics
2. Why does the cascade structure shown in Figure 5-1 make sense physically?

6

State Space Models

The objective of this chapter is to develop the linear state space models for the longitudinal and lateral dynamics of the hummingbird. This state space model will be used in later labs as we implement state space control methods.

6.1 Linear State Space Model for Longitudinal Dynamics

The linear equations of motion for the longitudinal dynamics are given by

$$\ddot{\theta} = b_\theta \tilde{F},$$

where we assume that $\tilde{F} = F_{fl} - F$ is the feedback linearized force.

Defining the state as

$$x_{lon} \triangleq (\theta, \dot{\theta})^T$$

and the deviated control input as

$$\tilde{u}_{lon} = \tilde{F},$$

then the linear state model is given by

$$\dot{x}_{lon} = A_{lon} x_{lon} + B_{lon} \tilde{u}_{lon},$$

If the output is $y_{lon} = \theta$, then the output equation is

$$y_{lon} = C_{lon} x_{lon}.$$

Note that if we design a controller law $\tilde{F} = \tilde{u}_{lon}(x_{lon})$, then the commanded force will be

$$F = F_{fl} + \tilde{u}_{lon}.$$

6.2 Linear State Space Model for Lateral Dynamics

From Equation (4.8), the linearized equations for the lateral dynamics are given by

$$\ddot{\tilde{\phi}} = \frac{1}{J_{1x}}\tilde{\tau} \tag{6.1}$$

$$\ddot{\tilde{\psi}} = \frac{1}{J_T + J_{1z}}\ell_T F_e \tilde{\phi}. \tag{6.2}$$

Define the lateral state as $x_{lat} = (\tilde{\phi}, \tilde{\psi}, \dot{\tilde{\phi}}, \dot{\tilde{\psi}})^\top$, and the lateral input $u_{lat} = \tilde{\tau}$. The lateral output is $y_{lat} = (\tilde{\phi}, \tilde{\psi})^\top$. Then the linearized state equations are given by

$$\dot{x}_{lat} = A_{lat}x_{lat} + B_{lat}u_{lat} \tag{6.3}$$

$$y_{lat} = C_{lat}x_{lat}. \tag{6.4}$$

Lab Assignment H.6

1. For the longitudinal dynamics, find A_{lon}, B_{lon}, and C_{lon} and pass them off to a TA.
2. For the lateral dynamics, find A_{lat}, B_{lat}, and C_{lat} and pass them off to a TA.

Part III

PID Control Design

7

Pole Placement for Second Order Systems

The objective of this lab assignment is to design a PD controller for the longitudinal dynamics of the hummingbird (i.e. pitch control). There are three main steps to this lab:

Step 1. Understand the transfer function for the longitudinal dynamics and use it to design the digital PD controller (i.e. finding the gains for k_p and k_d).

Step 2. Implement the design in your simulation and make sure that it performs as expected.

Step 3. Constrain the roll and yaw motion on the hummingbird and re-tune the PD gains on hardware, adjusting the gains to get satisfactory performance using the provided GUI.

The block diagram for the longitudinal controller is shown in Figure 7-1.

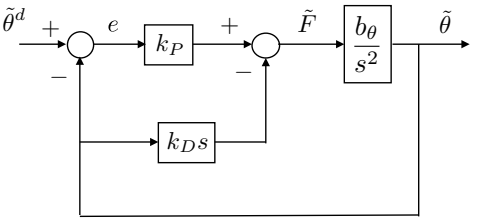


Figure 7-1: PD control design example.

A feedback linearizing force that cancels the gravity term can be calculated as

$$F_{fl} = (m_1\ell_1 + m_2\ell_2)\frac{g}{\ell_T}\cos\theta.$$

Remember that the control force sent to the propeller motors is given by $F = F_{fl} + \tilde{F}$, where $\tilde{F}$ is calculated from a PD controller as shown in Figure 7-1.

The closed-loop transfer function for the PD implementation shown in Figure 7-1 is

$$\frac{\tilde{\theta}(s)}{\tilde{\theta}^d(s)} = \frac{k_p b_\theta}{s^2 + k_d b_\theta s + k_p b_\theta}$$

where $\tilde{\theta}^d$ is the reference input or desired value for theta. The transfer function for a canonical second-order system can be expressed in terms of the damping ratio ζ_θ and the natural frequency ω_{n_θ} as

$$\frac{\omega_{n_\theta}^2}{s^2 + 2\zeta_\theta \omega_{n_\theta} s + \omega_{n_\theta}^2}.$$

Values for ζ_θ and ω_{n_θ} are determined from the desired transient response characteristics of the system. The natural frequency and desired rise time are approximately related by the expression

$$t_r \approx \frac{2.2}{\omega_{n_\theta}},$$

assuming that $\zeta_\theta \approx 0.707$. Alternatively, we can also use the following relationship for rise time and the natural frequency if we want to use a different value for ζ_θ:

$$t_r \approx \frac{1}{2} \frac{\pi}{\omega_n \sqrt{1 - \zeta^2}}.$$

Control gains are then calculated by equating the coefficients of the desired characteristic equation

$$s^2 + 2\zeta_\theta \omega_{n_\theta} s + \omega_{n_\theta}^2 = 0$$

with those of the characteristic equation of the closed-loop system, in this case

$$s^2 + k_d b_\theta s + k_p b_\theta = 0.$$

Complete the steps listed in the assignment below:

Lab Assignment H.7

1. Plug in actual values to find the transfer function from $\tilde{\Theta}^d$ to $\tilde{\Theta}$ of the closed loop system shown in Figure 7-1.
2. Find the characteristic polynomial $\Delta_{cl}(s)$ of the closed loop system.
3. Find k_P and k_D so that the rise time is $t_r = 1.0\ s$ and the damping ratio is $\zeta = 0.707$. Make the rise time a tuning parameter in your code so that you can adjust it to get good performance. If you want the system to be more damped, feel free to also play with ζ (probably between 0.707 and 0.95).

4. Implement a *digital* PD controller in `longitudinal_pd_controller.py` (in `src/case_studies/H_hummingbird/`). Make a new simulation driver (`labs/h7_sim.py`) that uses this controller, similar to the other case studies. The reference input should be a square wave with 30 degree amplitude and a frequency of 0.1 Hz. Tune the gains (k_p, k_d, and the low-pass filter parameter σ) in simulation to get satisfactory results. Make sure to constrain the torque to always be zero in your simulation (so that roll and yaw do not change). Pass this off to a TA.

5. As you begin working with the hardware, fix the rolling and yawing motion of the hummingbird using two of the three screws. See Figure 7-2.

6. Using the hummingbird hardware guide (as linked on Learning Suite), calibrate, with the hummingbird level zero the encoders, and then re-tune the k_m value to improve your overall performance. When completed, you can now click the "Pitch" button (as shown in Figure 7-3 below).

7. The command for θ_r or θ^d will turn into a step command that oscillates with that magnitude every 10 seconds. You can put a value between 10 and 20 degrees for this lab. Start with the gains found in simulation. However, it is very likely these will not work. So set k_d to zero and slowly increase k_p until you get a good dynamic response (even if the oscillation is a bit too high). Introduce the k_d gain slowly to reduce the oscillation. You can tune both parameters iteratively until you are happy with the performance. You do not need to change k_i as that will be explored in a future lab. We expect that there will be some steady state error, but significant oscillation will not be passed off by a TA. When you are happy with the performance, pass it off to a TA.

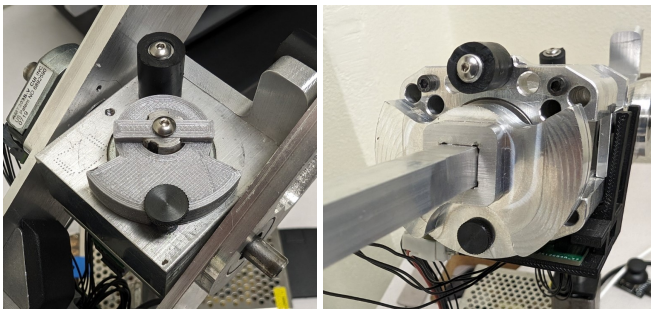


Figure 7-2: Close-up images of the two screws used to restrict yaw (shown on left) and roll (shown on right). Make sure the screws are centered and well-seated.

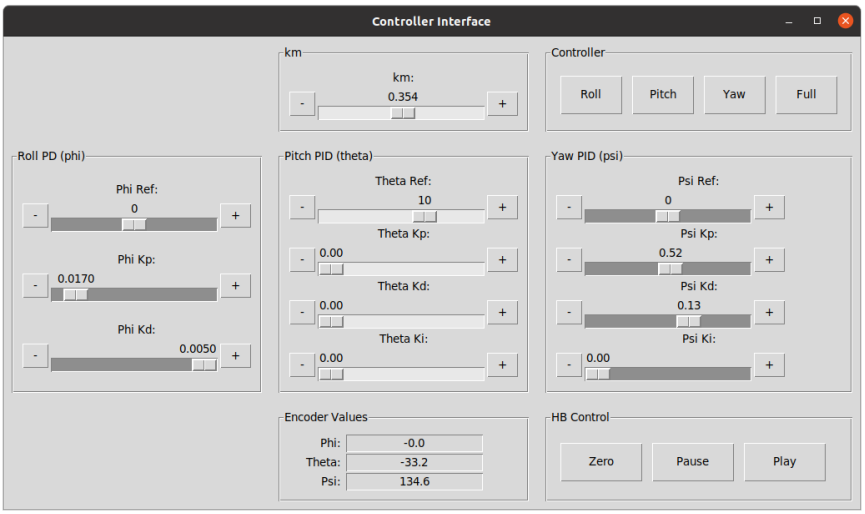


Figure 7-3: Screen-shot of the hummingbird GUI in “pitch” control mode.

Hints

- You should use the Python GUI code for the hardware as described in the guide on Learning Suite. It can be found in your own repository, or in the public repository at the following path:

  “byu_controlbook_public\labs\hardware”

- When you push the “play” button, the hummingbird will calibrate the motors the first time you turn it on. This is expected behavior, but don’t let it surprise you! And make sure to retune the k_m value for each hummingbird you use when you first start the GUI.

8

Design Strategies for Second-Order Systems

The objective of this lab assignment is to design the PD controller for the lateral dynamics of the hummingbird. There are four steps to this lab.

Step 1. Design the inner-loop/outer-loop control for the lateral dynamics.

Step 2. Implement the design in simulation and make sure that it performs as expected.

Step 3. Constrain the pitch and yaw motion on the hummingbird and implement the inner-loop control on roll in hardware, tuning the gains to get satisfactory performance.

Step 4. Remove the yaw constraint on the hummingbird, and implement the full lateral control, tuning the gains to get good performance.

Lateral Control Design

The lateral motion of the hummingbird is described by its rolling and yawing motion. Changes in the roll angle ϕ are caused by torques produced when the left and right thrust values are different. When ϕ is non-zero, a lateral force is produced on the hummingbird causing it to yaw, which is described by the yaw angle ψ. These lateral motions are modeled by the transfer functions derived in Section 5.2. The control strategy is successive loop closure as shown in Figure 8-1

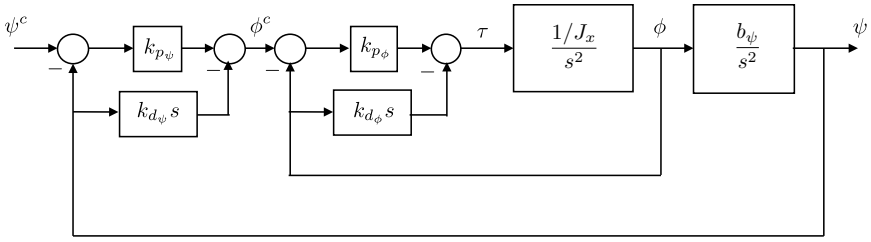


Figure 8-1: Successive loop closure block diagram for lateral dynamics.

Roll Loop Design

To design the lateral dynamics control system for the hummingbird using successive loop closure, we begin with the inner roll control loop. By design, the inner roll loop response should be roughly ten times faster than the outer yaw loop dynamic response. This will allow us to approximate the inner loop by its closed-loop DC gain when designing the outer loop.

The open loop transfer function of the inner roll loop is

$$\phi(s) = \frac{(1/J_{1x})}{s^2}\tau(s)$$

The closed-loop transfer function of the roll loop is

$$\frac{\phi(s)}{\phi^d(s)} = \frac{k_{p_\phi}/J_{1x}}{s^2 + \left(k_{d_\phi}/J_{1x}\right)s + \left(k_{p_\phi}/J_{1x}\right)} \tag{8.1}$$

with the corresponding characteristic equation

$$s^2 + \left(k_{d_\phi}/J_{1x}\right)s + \left(k_{p_\phi}/J_{1x}\right) = 0.$$

Based on the desired rise time t_{r_ϕ} and damping characteristics ζ_ϕ of the inner loop, we can determine the natural frequency of the roll dynamics ω_{n_ϕ} and the desired characteristic equation

$$s^2 + 2\zeta_\phi\omega_{n_\phi}s + \omega_{n_\phi}^2 = 0.$$

Proportional and derivative gains for the roll loop, k_{p_ϕ} and k_{d_ϕ} are determined by matching coefficients of the desired and actual characteristic equations.

Yaw Loop Design

Successive loop closure requires that there be adequate bandwidth separation between successive loops in the control design. The inner loop is designed to be the fastest (highest natural frequency), with each successive loop designed to be a factor of 10 to 20 slower in frequency. Higher natural frequencies correspond to shorter rise times. In the case of the yaw loop design, if we define M as the

bandwith separation factor (a constant between 10 and 20), and if the roll loop has a designed rise time of t_{r_ϕ}, then we would calculate the rise time of the slower outer yaw loop to be $t_{r_\psi} = M t_{r_\phi}$. Similarly, if the inner roll loop natural frequency is given by ω_{n_ϕ}, the outer yaw loop natural frequency would be calculated to be $\omega_{n_\psi} = \omega_{n_\phi}/M$. Bandwidth separation allows the inner loop dynamics to be replaced by the DC gain of the inner loop when designing the outer loop control, thus simplifying the outer-loop design significantly.

From the closed-loop transfer function for the roll loop given by equation 8.1, we can see that the DC gain of the inner loop is 1. Approximating the inner-loop dynamics with a transfer function of 1, the yaw closed-loop transfer function can be written as

$$\frac{\psi(s)}{\psi^d(s)} = \frac{k_{p_\psi} b_\psi}{s^2 + k_{d_\psi} b_\psi s + k_{p_\psi} b_\psi}$$

with the corresponding characteristic equation

$$s^2 + k_{d_\psi} b_\psi s + k_{p_\psi} b_\psi = 0.$$

As discussed above, the desired rise time t_{r_ψ} of the outer yaw loop is chosen to be a factor of 10 to 20 slower than the rise time of the inner roll loop. The yaw loop natural frequency ω_{n_ψ} is calculated from the desired yaw loop rise time. The damping ratio of the yaw loop ζ_ψ is chosen to give the desired overshoot and settling characteristics. With values for ω_{n_ψ} and ζ_ψ determined, the desired characteristic equation for the yaw loop can be expressed as

$$s^2 + 2\zeta_\psi \omega_{n_\psi} s + \omega_{n_\psi}^2 = 0.$$

Proportional and derivative gains for the yaw loop, k_{p_ψ} and k_{d_ψ} are determined by matching coefficients of the desired and actual characteristic equations. With the gains for both the roll and yaw loops calculated, the lateral successive loop closure design is complete.

Lab Assignment H.8

1. Implement the full lateral controller (inner loop for roll and outer loop for yaw) in simulation. We suggest using the pitch controller designed in the previous lab as a starting point. We need to include the pitch controller as well, because without longitudinal (or pitch) control, θ will not stay at zero as expected. Name the file "full_pd_controller.py" to match the expected naming in "__init__.py".

   Tune the rise time on yaw t_{r_ψ}, the bandwidth separation M to help pick the rise time on roll t_{r_ϕ}, and the two damping coefficients ζ_ψ and ζ_ϕ to get good response for the hummingbird in simulation.

2. For the hardware portion, you can start by running the GUI, as described in the previous lab and shown in Figure 7-3. We recommend also starting by tuning the k_m gain (with yaw and roll constrained).

3. After finding an appropriate k_m value, constrain the yaw and pitch motion on the hummingbird as shown in Figure 8-2.

4. To tune the roll controller first, you can click the "Roll" button and set a reference angle for ϕ. Whatever angle you choose will be the magnitude of a square wave that has a frequency defined in the parameter file. Beginning with the gains found in simulation (or beginning with zero values for all gains), tune the PD controller for roll to get an adequate response. You don't want the system to be too aggressive (so that it hits the end stops and bounces), but you do want it to be as quick and responsive as possible.

5. When the roll controller is well-tuned, remove the pin constraining the yaw motion, push the button for "Yaw" in the GUI and begin tuning the PD gains for ψ. This will cause your setting for the ϕ input to be ignored while you are tuning the PD controller for ψ. Beginning with the gains found in simulation (or beginning with zero gains), tune PD gains to get an adequate response.

6. You can iterate between the roll and yaw gains as needed, but please be careful and safe as you interact with the hummingbird. Remember it is OK (even preferred) to push the e-stop if the hummingbird is behaving poorly. You should at least stop the controller using the "Pause" button on the GUI if it is not working well. But when you switch it back on, and press the "Play" button on the GUI, the ESCs may recalibrate. Make sure to also check that your encoders are reading correct values, and if not, you can put the hummingbird in the zero position (straight out with zero pitch, and in the middle of its yaw range) and press the "Zero" button.

7. Pass off the simulation and the hardware results to the TA.

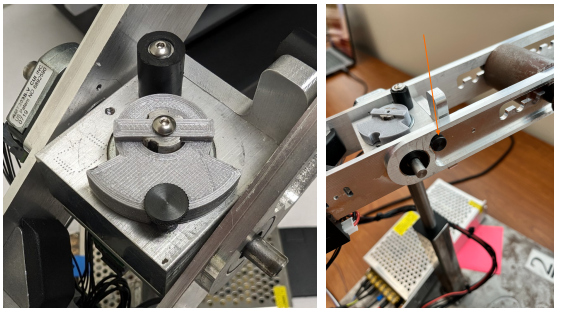


Figure 8-2: Close-up images of the two screws used to restrict yaw (shown on left) and pitch (shown on right). Make sure the screws are centered and well-seated.

Hints

- Sometimes, it is helpful to tune the inner loop first, and then the outer loop for successive loop control. However, to tune the roll controller, there is no easy way to constrain the pitch and yaw motion in the simulator (although if you want to try, you can constrain them by setting the second derivatives in your dynamics file to zero which will cause the system to not move in those directions - $\ddot{\theta}$ and $\ddot{\psi}$).
- Do you find something particularly difficult about this lab, and did you figure something out that you wish you would have known before starting the lab? Suggest a hint to the TAs or course instructor!

9

System Type and Integrators

Lab Assignment H.9

For the following, you can assume that we are using an unmodified PD controller to perform this analysis.

1. When the inner loop lateral controller for the hummingbird is PD control, what is the system type with respect to tracking of the inner loop? Characterize the steady state error when $\tilde{\phi}^r$ is a step, a ramp, and a parabola. What is the system type with respect to an input disturbance?

2. When the outer loop lateral controller for the hummingbird is PD control, what is the system type with respect to tracking of the outer loop? Characterize the steady state error when $\tilde{\psi}^r$ is a step, a ramp, and a parabola. What is the system type with respect to an input disturbance?

3. When the longitudinal controller for the hummingbird is PD control, what is the system type with respect to tracking of the longitudinal loop? Characterize the steady state error when $\tilde{\theta}^r$ is a step, a ramp, and a parabola. What is the system type with respect to an input disturbance?

10

Digital Implementation of PID Controllers

The purpose of this lab is to see the difference it makes when our model has some error and to also implement integral control on the hummingbirds. We will combine the longitudinal and lateral controllers and tune the gains to get satisfactory performance, and zero steady-state error.

Lab Assignment H.10

Simulation:

1. For simulation, start by adding 20% uncertainty to the model parameters. If you are using the provided solution for the `dynamics.py` file, you can simply declare your dynamics as follows:
   `hummingbird = HummingbirdDynamics(alpha=0.2)`

   Or you can implement this as you wish, but you will need to show the TAs how you implemented it.

2. In addition, add a disturbance step input of 0.1 N in force (or 0.05 N on each motor).

3. Try running your PD controller from the prior lab on this perturbed system. You should see some steady-state error in pitch due to the dynamic model now having mass parameters that are different from those used to calculate your feedforward force command.

4. Starting with the same PD gains that you used in the previous labs. Add a digital integral term for your longitudinal controller and an integral term on your outer loop for the lateral controller as well. If saturation occurs, tune your rise time parameters again to reduce the control gains so that saturation is eliminated. With your integrator implementation, implement

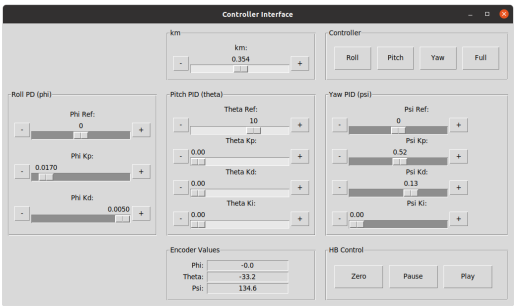


Figure 10-1: GUI used for lab H.10

an anti-windup scheme of your choice. *Pass off your simulation with the TA before proceeding to the next step.*

Hardware:

1. Run the file `hummingbirdLabs.py` and tune the gains for PID control for both lateral and longitudinal controllers on the hummingbird hardware. The GUI should look like the one shown in Figure 10-1. There is no need to add disturbances since the real system tends to have plenty of errors in your model and plenty of disturbance inputs. The following process may help you in tuning the controllers:

   (a) Constrain the roll and yaw, and tune k_m.

   (b) Now click the “Pitch" button and tune the PID gains for the longitudinal controller (or the controller for pitch). When it is working well.

   (c) Now press the “Pause” button and then constrain the pitch and release the roll. Press “Play” and “Roll,” and then re-tune the values for the PD gains for roll. When you are satisfied, pause the GUI again.

   (d) Next, release yaw, click “Play” and “Yaw,” and tune the values for the PID gains. When satisfied, pause the GUI.

   (e) Finally, release the pitch as well, click “Play” and “Full,” and adjust all gains as needed to get satisfactory performance for both controllers.

   (f) Congratulations!!

2. *Please remember to be cautious and careful with the hardware. It is somewhat fragile and could be dangerous, so please do not allow the hummingbird to go out of control! And please do not work with the hardware in the lab by yourself!*

Pass off your hardware implementation to the TA showing your ability to control both the pitch and the yaw of the hummingbird.

Hints

- For any tuning you do (for example after tuning k_m or $k_{p,\theta}$), you can click the “Pause" button and save the value for that parameter in `params.py` (near the top of the file). This will make it so that if you stop the GUI, and then restart it, your parameters will be the new starting defaults (rather than needing to re-tune everything).

Part IV

Observer-based Control Design

11

Full-state Feedback with Integral Control

The purpose of this lab is to design and implement full-state feedback control strategies with integral action (i.e. using an integrator).

11.1 Lateral Control using Linear State Feedback

The objective of lateral control is to design a linear state feedback controller of the following form:

$$\tau = \tau_e - K_{lat}(x_{lat} - x_{lat,e}) - k_{I,lat}x_{I,lat}$$

where $x_{I,lat}$ is defined as the integral of the error in yaw (i.e. $\int(\psi_r - \psi)dt$). These gains place the poles of the lateral dynamics at the desired pole locations AND places a pole for the integrator used in the lateral controller.

The first term, τ_e is the equilibrium torque found in Chapter 4, which is zero. The second term is the state feedback control based on the linearized or tilde state. The third term is the integral term that ensures that we have a steady state error of zero and that the DC-gain of the closed-loop system is one. If the equilibrium states are zero, $x_{lat,e} = (\phi_e, \psi_e, \dot{\phi}_e, \dot{\psi}_e)^\top = (0,0,0,0)^\top$, then the controller simplifies to the following:

$$\tau = -K_{lat}x_{lat} - k_{I,lat}x_{I,lat}$$

Gain values can be calculated with the `place` command in Python using the A_{lat}, B_{lat} matrices, along with the appropriate $C_{r,lat}$ matrix, and the desired pole locations. Suppose that we desire to tune the controller using the two natural frequencies ω_{n_ϕ} and ω_{n_ψ}, and the damping ratios ζ_ϕ and ζ_ψ, then the desired closed-loop poles are given by

$$\begin{aligned} p_{lat} &= [p_1, p_2, p_3, p_4, p_{I,lat}] \\ &= \left[-\zeta_\phi\omega_{n,\phi} \pm j\omega_{n,\phi}\sqrt{1-\zeta_\phi^2},\ -\zeta_\psi\omega_{n,\psi} \pm j\omega_{n,\psi}\sqrt{1-\zeta_\psi^2},\ -\omega_{n,\psi}/2\right] \end{aligned}$$

where we are setting the desired pole value for our integrator ($p_{I,lat}$) to $-\omega_{n,\psi}/2$ as a good starting point. However, you may define your poles in any way you choose to provide good performance.

To place the poles, you will also need to formulate the augmented A and B matrices for full state feedback control with integration (these are labeled A_1 and B_1 in the class textbook and notes). In Python, after having imported the correct module ("import control as cnt") the command

```
K_1_lat = cnt.place(A_1_lat, B_1_lat, p_lat);
```

can be used to calculate the lateral state feedback gains. Remember that when we implement the equation for torque shown above, $k_{I,lat}$ is the last entry in $K_{1,lat}$ (corresponding to the pole we defined for integration, $p_{I,lat}$) and will multiply the integral of the error in yaw ($x_{I,lat}$). While K_{lat} is the first four entries in $K_{1,lat}$.

11.2 Longitudinal Control using Linear State Feedback

The form of the controller for the longitudinal portion of the system is

$$F = F_e - K_{lon}(x_{lon} - x_{lon,e}) - k_{I,lon}x_{I,lon}$$

where $x_{I,lon}$ is defined as the integral of the error in pitch (i.e. $\int(\theta_r - \theta)dt$).

The first term is the feedforward equilibrium force found in Chapter 4. The second term is the state feedback control based on the linearized state. The third term is the integral term that ensures that we have a steady state error of zero and that the DC-gain of the closed-loop system is one. If the equilibrium states are zero, $x_e = (\theta_e, \dot{\theta}_e)^T = (0,0)^T$, then the controller simplifies to

$$F = F_e - K_{lon}x_{lon} - k_{I,lon}x_{I,lon}$$

The control design requires the placement of three poles (two for the longitudinal dynamics, and one for the integrator). Assuming that the desired closed-loop poles are characterized by a natural frequency ω_{n_θ} and damping ratio ζ_θ, the poles are given by

$$\begin{aligned} p_{lon} &= [p_1, p_2, p_{I,lon}] \\ &= \left[-\zeta_\theta\omega_{n,\theta} \pm j\omega_{n,\theta}\sqrt{1-\zeta_\theta^2},\ -\omega_{n,\theta}/2\right]. \end{aligned}$$

where we are setting the desired pole value for our integrator ($p_{I,lon}$) to $-\omega_{n,\theta}/2$ as a good starting point.

Again, after forming the augmented state space matrices ($A_{1,lon}$ and $B_{1,lon}$), we can use the Python command

```
K_1_lon = cnt.place(A_1_lon, B_1_lon, p_lon);
```

to calculate the longitudinal state feedback gains.

Remember that when we implement the equation for force shown above, $k_{I,lon}$ is the last entry in $K_{1,lon}$ (corresponding to the pole we defined for integration, $p_{I,lon}$) and will multiply the integral of the error in pitch ($x_{I,lon}$). While K_{lon} is the first two entries in $K_{1,lon}$.

Lab Assignment H.11

1. Carefully read the previous two sections and make sure you understand every line equation.

2. Augment the lateral and longitudinal state-space representations to include a state that is the integral of the output error. For the lateral dynamics, this is the integral of the yaw error and for the longitudinal dynamics, this is the integral of the pitch error.

3. Using the python control library (which we have used extensively now in the homework), compute the lateral gains K_{lat} and $k_{lat,I}$ using the `place` command, where the desired closed-loop pole locations are defined by ω_{n_ϕ}, $\omega_{n,\psi}$, ζ_ϕ, ζ_ψ, and the integral pole location. As a good starting point, as mentioned in the lab description, let the integral pole location be $-\omega_{n,\psi}/2$.

4. Also compute the longitudinal gains K_{lon}, $k_{lon,I}$ using the `place` command, where the desired closed-loop pole locations are defined by $\omega_{n,\theta}$, ζ_θ, and the integral pole location. As a good starting point, as mentioned in the lab description, let the integral pole location be $-\omega_{n,\theta}/2$.

5. Wherever you calculate your gains, make sure to also verify that the systems are controllable. If you're doing this in Python, this link is the function for checking controllability.

6. Implement the longitudinal and lateral full state feedback controllers with integral feedback and verify that the controller performs well in the simulation. Introduce a constant disturbance to the longitudinal control (e.g., an error in the equilibrium force) and a small constant disturbance torque to the yaw control and verify that they are mitigated by the integral control. This can be done by adding some constant value to your calculated force and torque in your controller code. Tune the controllers by adjusting values for damping ratio, natural frequency, and the integral pole (or by adjusting pole values directly) until you are satisfied with the overall performance.

12

Observers

Lab Assignment H.12.a

In this lab, we will be designing and implementing a full-state estimator, sometimes called a Luenberger observer, using pole placement methods. Your design will assume that only the angle states (roll, pitch, and yaw) are measured and available for feedback as the vector y. Full-state feedback requires both the angle and rate states (the derivatives of the angles), so we will use an estimator (or observer) to estimate the full state vector for the hummingbird.

1. Compute the lateral observer gains L_{lat} using the `place` command (in MATLAB or the python control library), where the desired observer pole locations are five times faster than the lateral controller pole locations (from Lab Assignment H.**??**).

2. Compute the longitudinal observer gains L_{lon} using the `place` command, where the desired observer pole locations are five times faster than the controller pole locations (from Lab Assignment H.**??**).

3. Using your `controller.py` file from the full-state feedback lab, implement a full-state estimator to estimate the state vector for the longitudinal and lateral dynamics of the hummingbird.

4. Implement full-state feedback controllers for longitudinal and lateral dynamics using estimated states for feedback.

5. Test your controller-observer system in simulation.

6. Implement the longitudinal and lateral observers on the hummingbird hardware in conjunction with the full-state feedback designs from a prior lab. Demonstrate their correct function. Tune estimator pole locations if needed.

Hints

- When you place your observer poles, remember to take the transpose of A and C. This is due to the mathematical duality between observability of (A, C) and controllability of (A^T, C^T).
- As with the other hummingbird labs, when debugging, it may be beneficial to consider the longitudinal and lateral systems separately. This can be done by zeroing the state derivatives in the `dynamics.py` file for those states that you want to ignore. You can then focus on the control and estimation for each subsystem separately before combining them together.

Lab Assignment H.12.b

This lab is designed to stretch your skills in ROS and controls. So, don't be afraid to ask for help from a TA. In this lab, we will be designing a Luenberger observer and you will be writing the ROS Node to implement it.

1. By hand, draw the ROS network for the combined controller-observer system, carefully identifying the topics and message types each node will publish and subscribe two. Before, proceeding, have a TA check off your network.
2. Using your `controller.py` (from Lab Assignment H.**??**) as a guide, create a new file called `observer.py`.
3. Make necessary changes to the node name so it doesn't conflict with `controller.py`.
4. Add in your publishers and subscribers to `observer.py`.
5. Set the publishers and subscribers in `observer.py`.
6. In `observer.py`, verify that the systems (A_{lat}, C_{lat}) and (A_{lon}, C_{lon}) are observable.
7. Make callbacks for the subscribers in `observer.py`. There should be 2 callbacks, one for each subscriber.
8. Compute the lateral observer gains L_{lat} using the `place` command, where the desired observer pole locations are five times faster than the lateral controller pole locations.
9. Compute the longitudinal observer gains L_{lon} using the `place` command, where the desired observer pole locations are five times faster than the controller pole locations.

TOC

10. Test your observer to make sure that it localizes correctly. Note: at this point if you run `rqt_graph` the system will not be closed loop, we are only testing to make sure we get a good estimate.

11. Have the TA check your `rqt_graph` and observer estimate to make sure your observer is working properly.

12. Modify `controller.py` to subscribe to the estimate coming out of your observer, and use that estimate for your control. Note: this means you should no longer be differentiating states to get velocity estimates.

13. Test your controller-observer system in simulation.

14. Implement the longitudinal and lateral observers on the hummingbird hardware in conjunction with the full-state plus integral feedback designs from the last lab. Demonstrate their correct function. Tune estimator pole locations if needed.

Hints

- When drawing your ROS network, you should only concern yourself with 3 nodes in your system. Don't worry about the nodes that publish to Rviz.
- For the publishers/subscribers associated with your state estimate $\hat{x}$, you can use `geometry_msgs/Twist`. Basically it contains 2 Vector3 attributes. You can store the roll, pitch and yaw in linear and the derivatives in angular .
- You do not need to make a separate node for the longitudinal and latitudinal observers. These should reside in the same node.
- It's best to set up `observer.py` with a class structure and store your state estimates as member variables.
- For help with setting the publishers and subscribers refer to `Controller.py` and rospy pub sub tutorial.
- When you place your observer poles, remember to take the transpose of A and C. This is due to the duality between observability of (A, C) and controllability of (A^T, C^T).
- `observer.py` should have 2 subscribers and 1 publisher. See Figure 13.1 in the controlbook.
- When making the callbacks, have the callback associated with u update some member variable and have the callback associated with y_m propogate the dynamics of your observer.
- Use RK4 to propogate the dynamics of your observer. You can modify the RK4 method in your simulator to accomplish this.

TOC

- You can test your observer estimation by adding in another publisher for the hummingbird estimate with message type `State.msg`. You can view this message through Rviz to see your state estimate. You can even make your own message based on the `State.msg` that includes the angle rates and call it `FullState.msg`. If you define this file in the same folder as `State.msg`, (/hb/hb_msgs/msg/) then you can simply run "catkin_make" in your main hummingbird workspace directory and it will generate code for this new message type.
- You can streamline testing your controller-observer system by adding an additional node tag to `controller_sim.launch` and `controller_hw.launch`.

13

Disturbance Observers

Lab Assignment H.13

1. Verify that the lateral and longitudinal systems are observable using the $control.obsv()$ command.

2. Compute the estimator gains L for the lateral and longitudinal systems using the $scipy.signalimportplace_poles$ command.

3. Implement the lateral and longitudinal controllers using the estimated states $xhat$. Verify that the controller meets the time rise requirements. Tune the gains if necessary.

13.0.1 Python*

4. Modify your main.py and sim_plot.py files to plot the estimated states along with the actual states. Tune the estimator gain as necessary so that the estimated states closely approximate the real states.

13.0.2 Matlab*

4. Modify your simulink to plot the estimated states along with the actual states. Tune the estimator gain as necessary so that the estimated states closely approximate the real states.

Hints

a

Parameters of the Hummingbird

The physical parameters of the hummingbird are given in the table below, where m_1, J_1 are the mass and inertia of the hummingbird head, rotors, and rolling shaft, m_2, J_2 are the mass and inertia of the pitching body, and m_3, J_3 are the mass and inertia of the hummingbird that only yaws. When the roll pitch and yaw angles are all zero, the position of the center of mass of m_1 is $(\ell_1, 0, 0)^\top$, the position of the center of mass of m_2 is $(\ell_2, 0, 0)^\top$, and the position of the center of mass of m_3 is $(\ell_{3x}, \ell_{3y}, \ell_{3z})^\top$. The distance from the center of rotation to the hummingbird head is ℓ_T and the distance from the center of the hummingbird head to the rotors is d.

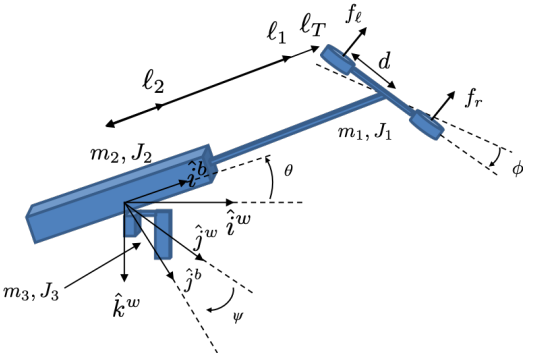


Figure 1-1: Hummingbird coordinate frames and nomenclature.

The hummingbird in the lab has the following parameters, which have been found experimentally:

The parameter value k_m is not given, but will be computed in one of the labs.

g	9.81	m/s^2
ℓ_1	0.247	m
ℓ_2	-0.039	m
ℓ_{3x}	-0.007	m
ℓ_{3y}	-0.007	m
ℓ_{3z}	0.018	m
ℓ_T	0.355	m
d	0.12	m
m_1	0.108862	kg
J_1	diag$(0.000189, 0.001953, 0.001894)$	kg-m^2
m_2	0.4717	kg
J_2	diag$(0.00231, 0.003274, 0.003416)$	kg-m^2
m_3	0.1905	kg
J_3	diag$(0.0002222, 0.0001956, 0.000027)$	kg-m^2
k_m	?	N/PWM